

COVID-19

MATH245a/b/c, 2025/26, Final Project 1

39588882

Mathematical model played a critical role during the COVID-19 pandemic. One of the simplest models for an infectious disease is the susceptible-infectious-removed (SIR) model, that divides the population in three groups. Susceptible individuals can become infected when they contact an infectious individual, and infectious individuals are removed either by recovering or by death.

Question 1

Assuming that the population is *closed*, we can consider only the susceptible and infectious compartments of the model. Let x be the fraction of the population in the susceptible compartment, and y the fraction in the infectious compartment. Note that the *closed* population assumption means that $1 - x - y$ is the fraction in the removed compartment. The SIR model ordinary differential equations are

$$\begin{aligned}x' &= -\beta xy, \\y' &= \beta xy - \gamma y,\end{aligned}$$

where β models the chances of infection when a susceptible meets an infectious individual, and γ is the recovery rate (how long it takes to recover or to die from the disease).

- (a) Implement a function `euler(beta,gamma,x0,y0,T,h)` that receives values for the model parameters, the initial values of the variables x, y and the final simulation time T (we are assuming the initial time is $t = 0$), and the step h , and applies the Euler method to solve the SIR system. Your function should return a dataframe with three columns, t, x and y . Test it with $x_0 = 0.9$, $y_0 = 0.1$, $\beta = 0.6$, $\gamma = 0.2$, $h = 10^{-1}$ and $T = 40$, and display the first few rows of the dataframe with `head`.

```

def euler(beta,gamma,x0,y0,T,h):
    """
    Solving the SIR using Euler's method.

    Arguments:
    f1,f2 : functions representing dx/dt,dy/dt
    x0,y0 : initial value
    h : step size
    t,x,y: list of all t,x,y values
    """
    t = np.arange(0,T+h,h)
    x = np.zeros(len(t))
    y = np.zeros(len(t))
    x[0] = x0
    y[0] = y0
    f1 = lambda x,y: -beta*x*y
    f2 = lambda x,y: beta*x*y - gamma*y
    for i in range(len(t)-1):
        x[i+1] = x[i] + h*f1(x[i],y[i])
        y[i+1] = y[i] + h*f2(x[i],y[i])
    data = {"t":t,"x":x,"y":y}
    return pd.DataFrame(data)

beta = 0.6
gamma = 0.2
x0 = 0.9
y0 = 0.1
T = 40
h = 0.1
#arguments are written out clearly for readability

df = euler(beta,gamma,x0,y0,T,h)
df.head()

```

	t	x	y
0	0.0	0.900000	0.100000
1	0.1	0.894600	0.103400
2	0.2	0.889050	0.106882
3	0.3	0.883348	0.110446
4	0.4	0.877495	0.114091

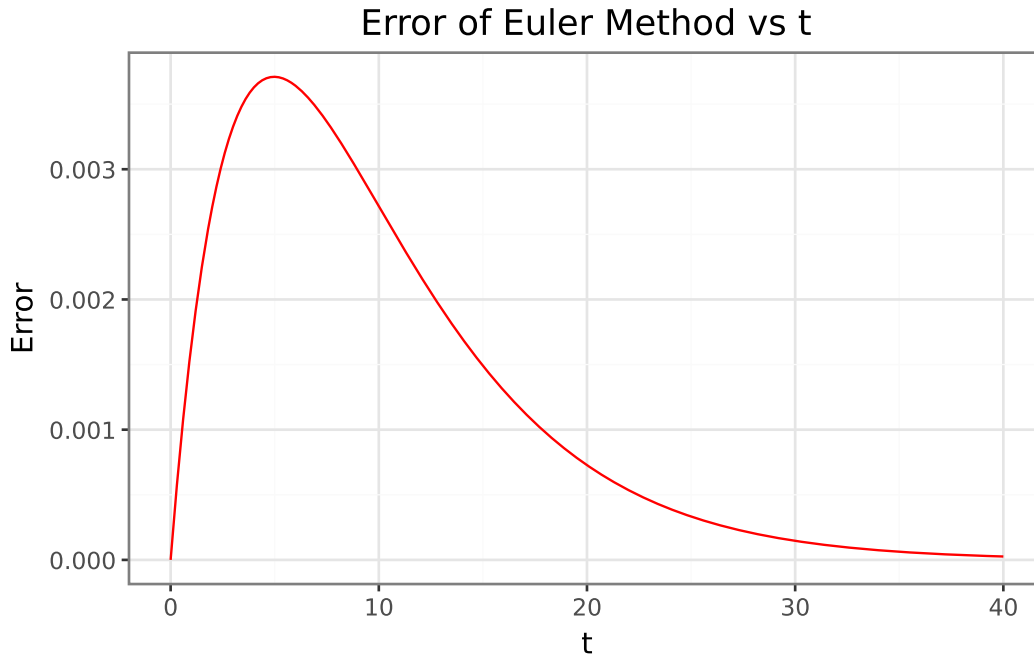
(b) If the fraction of susceptibles is 0, the equation for the infectious can be solved analytically

(it is an exponential function). Solve the SIR model with $x_0 = 0$, $y_0 = 1$, $\beta = 0.6$, $\gamma = 0.2$, $h = 10^{-1}$ and $T = 40$, and add a column `exact` to the dataframe produced by the function `euler` with the exact solution at the corresponding time t . Plot the difference between the exact solution and the Euler method as a function of time, using `geom_line`.

```
def exact(beta,gamma,T,h):
    #Analytic solution to the SIR model under the condition x0 = 0
    #The SIR model becomes independent of x
    """
    Solving the SIR analytically
    Arguments:
    h : step size
    t,y: list of all t,y values
    """
    t = np.arange(0,T+h,h)
    Y = np.exp(-gamma*t)
    return Y

beta = 0.6
gamma = 0.2
x0 = 0
y0 = 1
T = 40
h = 0.1
df = euler(beta,gamma,x0,y0,T,h)
Y = exact(beta,gamma,T,h)

vertical = np.abs(df["y"].to_numpy()-Y)
plot = {"t":df["t"],"y":vertical}
plot = pd.DataFrame(plot)
a = (
    pn.ggplot(plot, pn.aes(x="t", y="y"))
    + pn.geom_line(color = "red")
    + pn.theme_minimal() + pn.ylab("Error") + pn.theme_bw()
    + pn.labs(title="Error of Euler Method vs t")
)
a
```



- (c) If we are interested on the weekly cases, it might seem reasonable to use a time step h of a week. Use the `euler` function to solve the SIR model with the same parameters and initial values as in Question 1a, but use $h = 7$ days. Plot x and y as a function of time, in the same plot with two different colours. Write one or two sentences to give an explanation of your findings.

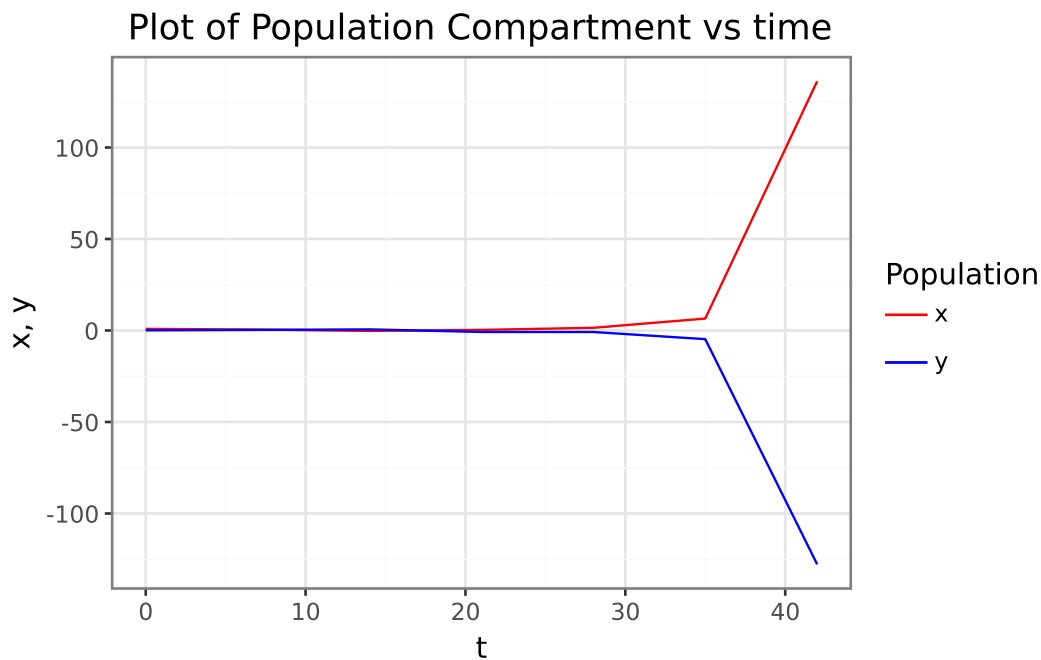
```
beta = 0.6
gamma = 0.2
x0 = 0.9
y0 = 0.1
T = 40
h = 7
#solve SIR using euler method
df = euler(beta,gamma,x0,y0,T,h)
#prepare data for plotting
Plot = pd.DataFrame({"t":df["t"],"x":df["x"],"y":df["y"]})

b = (
    pn.ggplot(Plot, pn.aes(x='t'))
    + pn.geom_line(pn.aes(y='x', color='x'))
    + pn.geom_line(pn.aes(y='y', color='y'))
    + pn.theme_bw()
    + pn.scale_color_manual(values={
```

```

    "x": "red",
    "y": "blue"})
+ pn.labs(
    title="Plot of Population Compartment vs time",
    color="Population"
)
+ pn.ylab("x, y")
)
b #display plot

```



Remark 1 The step size of $h = 7$ is too large for the euler method to converge. The values of x and y blow up and are not valid for values of $h \geq 28$ ($0 \leq x, y \leq 1$).

Question 2

The data in `omicron.csv` contains the number of *new* COVID-19 infected individuals during the wave caused by the Omicron variant of the virus. It useful to note that the Office for National Statistics population estimate for England from the 2021 census is 56,490,048.

In the SIR model $x(t)$, $y(t)$ and $z(t)$ are the proportions of individuals in each compartment at any given time. In this model infected individuals remain in the $y(t)$ compartment for several days. After being infected for several days individuals recover and move into the recovered

compartment $z(t)$. It is important to realise that $y(t)$ is **not** the proportion of new infected individuals per time increment (h). If $h = 1$ day then the proportion of new infected individuals (that we will call *incidence*) per day is $y(kh) - y((k-1)h)$ where $k = 1, 2, 3, \dots$

- (a) Load `omicron.csv` into a dataframe, and add columns to the dataframe to represent time in days (the first row being $t = 1$) in a column `t`, and the number of new cases expressed as a fraction of the total population in a column called `incidence`. Plot `incidence` vs `t`.

```
England = 56490048.0 # England population
Omicron = pd.read_csv("omicron.csv") # Load data
t = np.arange(1,len(Omicron)+1) #time variable
Omicron["t"] = t
Omicron["incidence"] = Omicron["cases"]/England
# Plot
s = (
    pn.ggplot(Omicron,pn.aes(x="t",y="incidence"))
    +pn.theme_bw() + pn.geom_line()
    + pn.labs(title="Plot of New Cases vs Time")
)
s
```



- (b) Implement a function `incidence(beta,gamma,x0,y0,T,h)` that receives the same parameters as `euler` but returns a dataframe with two columns, time t and the corresponding

incidence at each time. Test it with $x_0 = 0.9$, $y_0 = 0.1$, $\beta = 0.6$, $\gamma = 0.2$, $h = 10^{-1}$ and $T = 40$, and display the first few rows of the dataframe with `head`.

```
def incidence(beta,gamma,x0,y0,T,h):
    t = np.arange(0,T+h,h) #time
    x = np.zeros(len(t)) #x vales
    y = np.zeros(len(t)) #y values
    incident = [0]*len(t) #incidence values
    incident[0] = np.nan #no incidence at t=0
    x[0] = x0
    y[0] = y0
    f1 = lambda x,y: -beta*x*y
    f2 = lambda x,y: beta*x*y - gamma*y
    for i in range(len(t)-1):
        x[i+1] = x[i] + h*f1(x[i],y[i])
        y[i+1] = y[i] + h*f2(x[i],y[i])
        incident[i+1] = y[i+1]-y[i] # change in y = incidence
    incident = np.array(incident)
    # Return results as DataFrame
    data = {"t":t,"incidence":incident}
    return pd.DataFrame(data)

#Arguments
beta, gamma = 0.6, 0.2
x0, y0 = 0.9, 0.1
T, h = 40, 0.1

table = incidence(beta,gamma,x0,y0,T,h) #run model
table.head()
```

	t	incidence
0	0.0	NaN
1	0.1	0.003400
2	0.2	0.003482
3	0.3	0.003564
4	0.4	0.003645

- (c) Write a function factory to fix the values of `gamma,x0,y0,T,h`. Use it to plot the incidence for $\beta = 0.5, 0.6, 0.7$ as a function of time, for $\gamma = 0.2$, $h = 10^{-1}$, $x_0 = 0.9$, $y_0 = 0.1$ and $T = 40$. Please plot the three values of β in the same plot, with different colours.

```

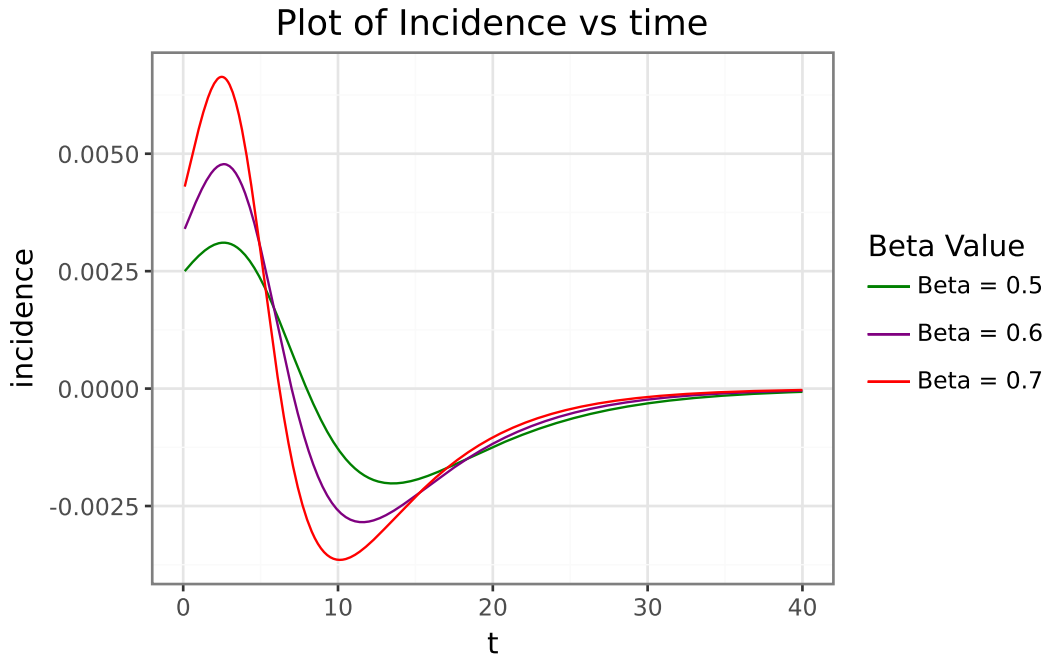
# Return a function with fixed arguments except beta
def fix(gamma,x0,y0,T,h):
    g = lambda beta : incidence(beta,gamma,x0,y0,T,h)
    return g

beta = [0.5,0.6,0.7] # beta values to compare
another_plot = [] #list of lists containing the incidences for each beta value
a = fix(0.2,0.9,0.1,40,1.e-1) # Fix arguments except beta
for b in range(len(beta)): # Generate incidence curves for each beta
    x = a(beta[b])
    another_plot.append(x)
(# Plot all curves on same graph
    pn.ggplot()

    + pn.geom_line(another_plot[0],
    pn.aes("t", "incidence", color="Beta = 0.5"))
    + pn.geom_line(another_plot[1],
    pn.aes("t", "incidence", color="Beta = 0.6"))
    + pn.geom_line(another_plot[2],
    pn.aes("t", "incidence", color="Beta = 0.7"))

    # Manually set colors
    + pn.scale_color_manual(values={
        "Beta = 0.5": "green",
        "Beta = 0.6": "purple",
        "Beta = 0.7": "red"
    })
    + pn.labs(title="Plot of Incidence vs time", color="Beta Value")
    + pn.theme_bw()
)

```

Question 3

This question will use the real data in the file `omicron.csv`.

- (a) Create a function `error(beta,gamma,x0,y0,data)` that receives the same parameters as the function `euler` and a dataframe `data` that contains a column `t` and a column `incidence` (the incidence in the real data), and returns the squared difference $\sum (m_t - d_t)^2$, where m_t is the incidence from the model solution and d_t is the incidence in the data, and the sum is over all the `data` points t . For the Euler method, use the step h at which the data is defined (i.e. 1 day), and similarly use the maximum time T from the data. Test your function with $\beta = 0.6$, $\gamma = 0.1$, $x_0 = 0.9$, $y_0 = 0.1$, and print the result.

```
def error(beta,gamma,x0,y0,data):
    # squared error between model and data
    deviation = (incidence(beta,gamma,x0,y0,len(data)
    ,h=1)["incidence"].iloc[1:].reset_index(drop=True)
    -data["incidence"]).tolist()
    deviation = np.sum(np.square(np.array(deviation)))
    return deviation
```

```
beta = 0.6
gamma = 0.1
```

```
x0 = 0.9
y0 = 0.1

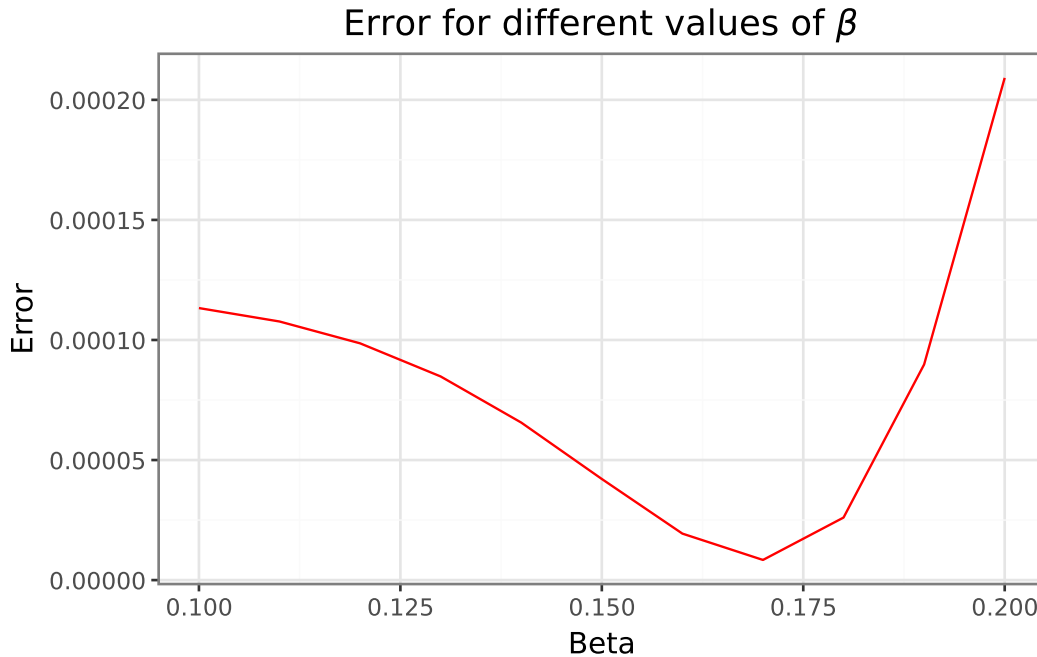
print(error(beta,gamma,x0,y0,Omicron))
```

0.049241996400050565

- (b) Implement a function factory `factory_error` to fix the values of `gamma,x0,y0,data`. Plot the error as a function of β for $\gamma = 0.1, x_0 = 1 - 0.004, y_0 = 0.004$, and $\beta = 0.1, 0.11, 0.12, \dots, 0.2$.

```
# Error function with beta as variable
def factory_error(gamma,x0,y0,data):
    def g(beta1):
        return error(beta1,gamma,x0,y0,data)
    return g

beta = np.arange(0.1, 0.21, 0.01) # beta range
gamma, x0, y0 = 0.1, 1 - 0.004, 0.004
# Compute errors
ERROR = factory_error(gamma, x0, y0, Omicron)
Error_array = [ERROR(b) for b in beta]
Error_array = pd.DataFrame({"Error":Error_array,"Beta": beta})
#plot
(
    pn.ggplot(Error_array, pn.aes("Beta","Error"))
    + pn.geom_line(color = "red")
    + pn.theme_bw() +pn.labs(
        title = r"Error for different values of $\beta$"
    )
)
```



- (c) Write a function factory `findmin_factory(gamma,x0,y0,data)` that returns `findmin(beta_min,beta_max,N)` to evaluate the error at N equidistant values of β between `beta_min` and `beta_max`. Use it to find the value of β that minimises the error for $\beta \in [0.1, 0.2]$, with $N = 100$, for $\gamma = 0.1$, $x_0 = 1 - 0.004$, $y_0 = 0.004$ and the omicron data.

```
def findmin_factory(gamma,x0,y0,data):
    def findmin(beta_min,beta_max,N):
        L = [] # store errors
        # grid of beta values
        BETA_Array = np.linspace(beta_min,beta_max,N+1)
        # compute error for each beta
        for i in range(len(BETA_Array)):
            L.append(ERROR(BETA_Array[i]))
        # return beta with smallest error
        index = np.argmin(L)
        return BETA_Array[index]
    return findmin

# Search interval and resolution
beta_min = 0.1
beta_max = 0.2
N = 100
```

```

# Fixed arguments
gamma = 0.1
x0 = 1-0.004
y0 = 0.004
data = Omicron
# Find optimal beta
X = findmin_factory(gamma,x0,y0,data)
Optimal_beta = X(beta_min,beta_max,N)

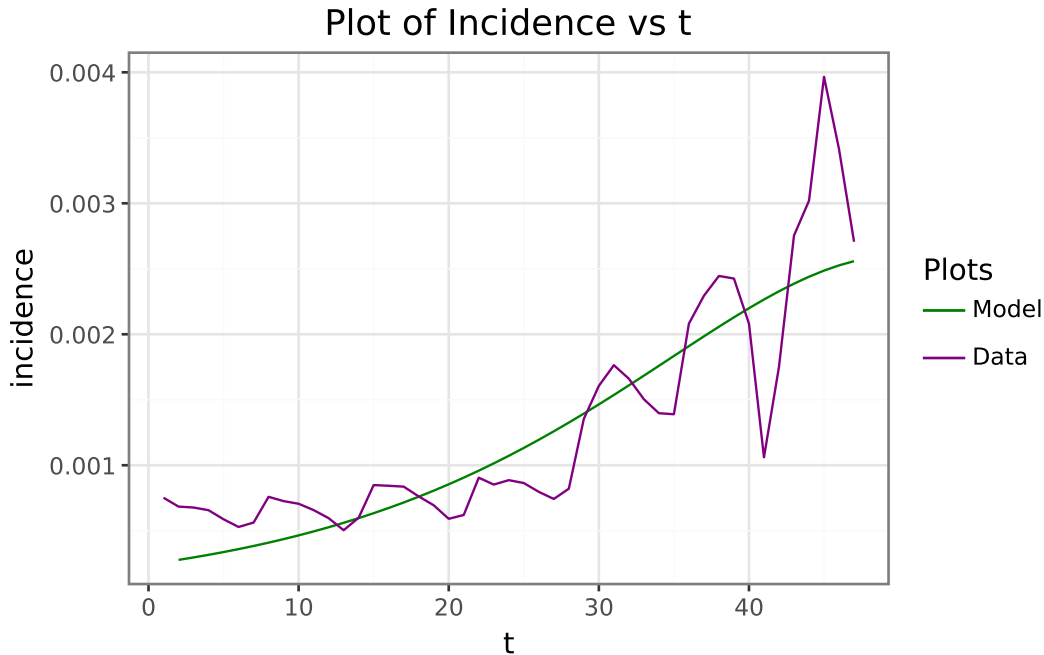
```

- (d) Plot the observed incidence and the model incidence with the optimal β , in one plot, using different colours for model and data.

```

fixed = fix(gamma,x0,y0,len(data)-1,h=1)
F = fixed(Optimal_beta)
F["t"] = np.arange(1,48,1) # align time with data
# Model vs data
(
    pn.ggplot()
    + pn.geom_line(F, pn.aes("t", "incidence", color="Model"))
    + pn.geom_line(Omicron, pn.aes("t", "incidence", color="Data"))
    + pn.scale_color_manual(values={
        "Model": "green",
        "Data": "purple"
    })
    + pn.theme_bw()
    + pn.labs(
        title="Plot of Incidence vs t",
        color="Plots"
    )
)

```



Question 4

The SIR model can be extended to include a vaccinated compartment.

$$\begin{aligned}x' &= -\beta xy - \mu x, \\y' &= \beta xy - \gamma y, \\v' &= \mu x,\end{aligned}$$

where μ is a vaccination rate. We will call this model SIRV.

- (a) Write a function `solve(beta,gamma,mu,x0,y0,v0,T,h)` to solve the SIRV model using `deSolve` (in R) or `scipy` (in Python). The function should return a dataframe with columns `t,x,y,v`, with the solution evaluate at times $0, h, 2h, \dots$. Test it by plotting the difference between the solution using `solve` with `mu=0` and the solution using `euler`, for $\beta = 0.15, \gamma = 0.1, x_0 = 1 - 0.004, y_0 = 0.004, v_0 = 0, T = 40, h = 1$. Plot only the difference between solutions for the variables x and y , each variable in a different colour.

```
# Solve SIRV using scipy.integrate
def solve(beta,gamma,mu,x0,y0,v0,T,h):
    t = np.arange(0,T+h,h)
    initial = [x0,y0,v0]
```

```

# System of ODEs
def system(vars,t):
    x,y,v = vars
    dxdt = -beta*x*y - mu*x
    dydt = beta*x*y - gamma*y
    dvdt = mu*x
    return [dxdt,dydt,dvdt]

# Solve system
solution = odeint(system,initial,t)
# Extract components
x = solution[:,0]
y = solution[:,1]
v = solution[:,2]
solution_df = pd.DataFrame({"t":t,"x":x,"y":y,"v":v})
return solution_df

#argument
beta = 0.15
gamma = 0.1
mu = 0
x0 = 1-0.004
y0 = 0.004
v0 = 0
T = 40
h = 1
sol = solve(beta, gamma, mu, x0, y0, v0, T, h)

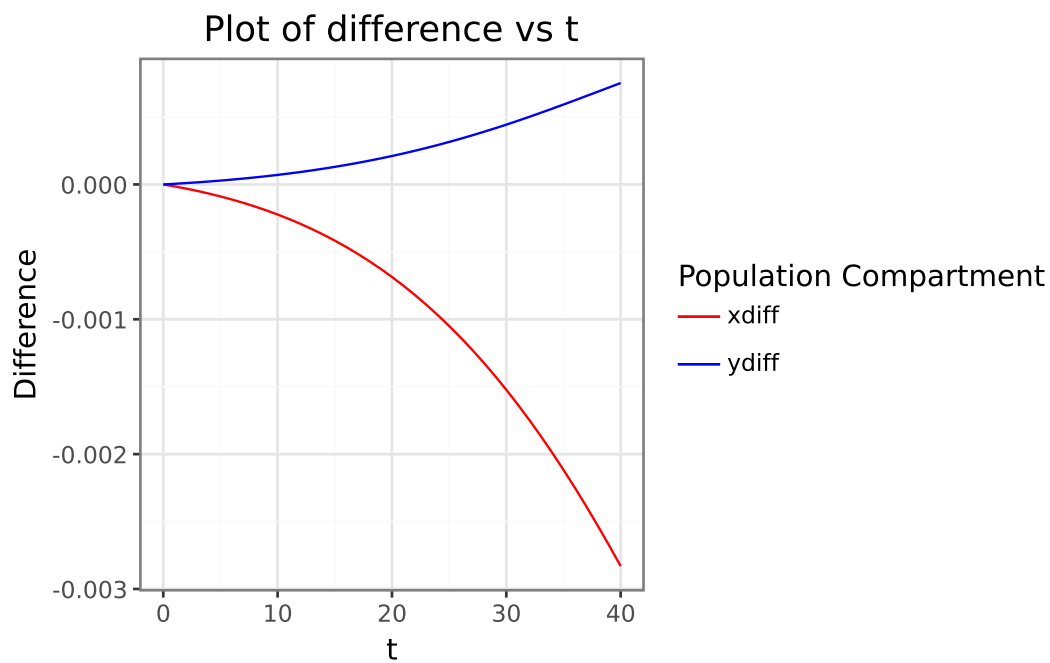
diff = {
    "xdiff": sol["x"].to_numpy() - euler(beta,gamma,x0,y0,T,h)["x"].to_numpy(),
    "ydiff": sol["y"].to_numpy() - euler(beta,gamma,x0,y0,T,h)["y"].to_numpy(),
    "t": sol["t"]}
}
diff = pd.DataFrame(diff)
# Plot differences
(
    pn.ggplot()
    + pn.geom_line(diff, pn.aes("t", "xdiff", color="xdiff"))
    + pn.geom_line(diff, pn.aes("t", "ydiff", color="ydiff"))
    + pn.scale_color_manual(values={
        "xdiff": "red",
        "ydiff": "blue"
    })
)

```

```

})
+ pn.theme_bw()
+ pn.labs(
  title="Plot of difference vs t",
  color="Population Compartment"
) + pn.ylab("Difference")
)

```



- (b) Plot $x, y, 1-x-y-v, v$, using different colours for each variable, using the same parameters as in Question 4a, with $\mu = 0.002$ and $T = 200$.

```

#parameters
beta = 0.15
gamma = 0.1
mu = 0.002
x0 = 1-0.004
y0 = 0.004
v0 = 0
T = 200
h = 1

#solve system

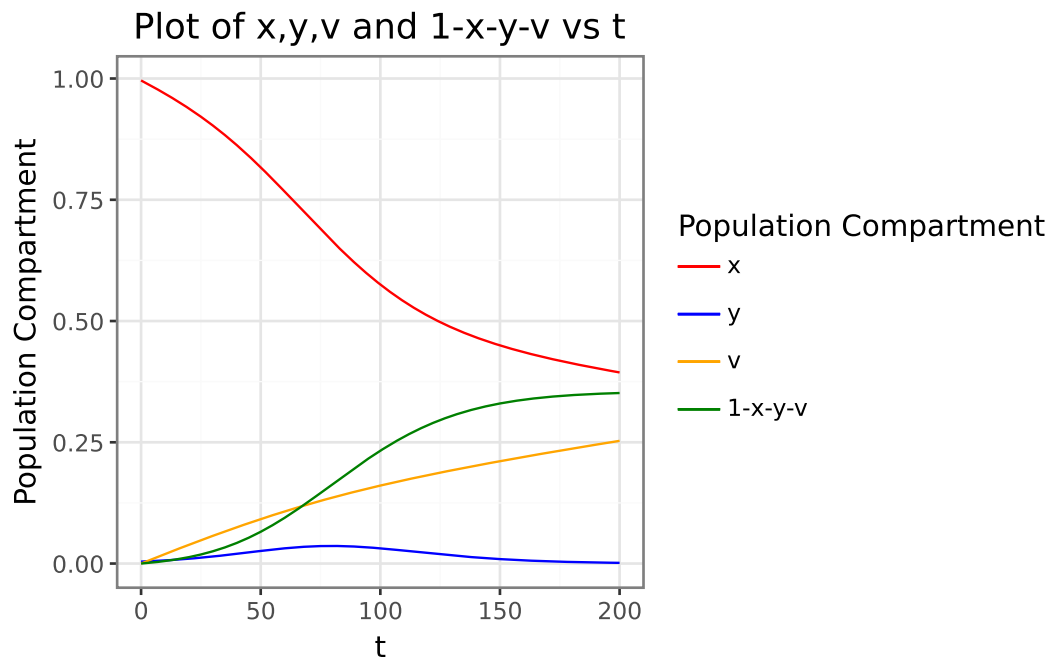
```

```

soln = solve(beta,gamma,mu,x0,y0,v0,T,h)
soln["1-x-y-v"] = 1-soln["x"]-soln["y"]-soln["v"]

(
  pn.ggplot()
  + pn.geom_line(soln, pn.aes("t", "x", color='x'))
  + pn.geom_line(soln, pn.aes("t", "y", color='y'))
  + pn.geom_line(soln, pn.aes("t", "v", color='v'))
  + pn.geom_line(soln, pn.aes("t", "1-x-y-v", color='1-x-y-v'))
  + pn.scale_color_manual(values={
    "x": "red",
    "y": "blue",
    "v": "orange",
    "1-x-y-v": "green"
  })
  + pn.theme_bw()
  + pn.labs(
    title="Plot of x,y,v and 1-x-y-v vs t",
    color="Population Compartment"
  ) + pn.ylab("Population Compartment")
)

```



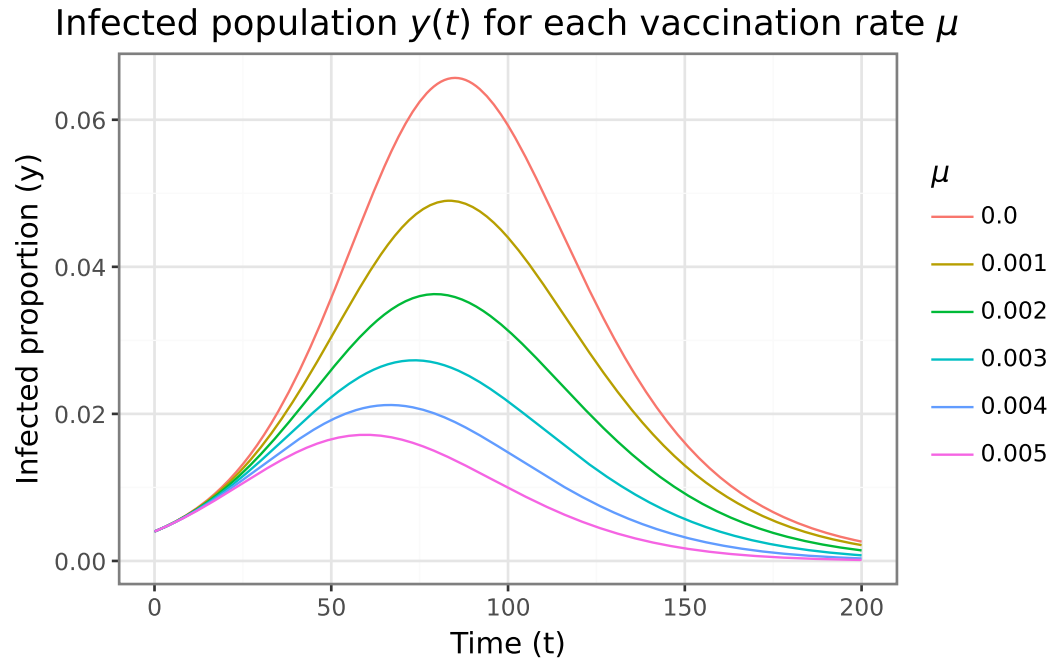
(c) Plot fraction of infectious individuals using the same parameters, with $\mu =$

0, 0.001, ..., 0.005. Write one or two sentences discussing the impact of vaccinations in view of this results.

```
#Arguments
beta = 0.15
gamma = 0.1
mu = np.arange(0,0.006,0.001) # vaccination rates
x0 = 1-0.004
y0 = 0.004
v0 = 0
T = 200
h = 1

# Solve model for each mu and store infected population
BigY = [solve(beta, gamma, m, x0, y0, v0, T, h)["y"] for m in mu]

# Combine results into DataFrame
Infection_Plot = dict(zip(mu, BigY))
Infection_Plot["t"] = np.arange(0, T+h, h)
Infection_Plot = pd.DataFrame(Infection_Plot)
# Convert to long format for plotting
Infection_Plot_Long = Infection_Plot.melt(
    id_vars=["t"],
    value_vars=mu,
    var_name="mu_value",
    value_name="y"
)
# Convert mu labels to strings
Infection_Plot_Long["mu_value"] = Infection_Plot_Long["mu_value"].astype(str)
# Plot infected population for different mu
(
    pn.ggplot(Infection_Plot_Long, pn.aes(x="t", y="y", color="mu_value"))
    + pn.geom_line()
    + pn.theme_bw()
    + pn.labs(
        title=r"Infected population  $y(t)$  for each vaccination rate  $\mu$ ",
        x="Time (t)",
        y="Infected proportion (y)",
        color=r" $\mu$ "
    )
)
```



Remark 2 For smaller values of μ , the infectious compartment of the population (y) peaks higher and grows faster while the opposite is the case for bigger values of μ which causes the peak values of y to be smaller and occur later.

We can conclude that higher vaccination rates help slow down the spread of the disease (Omicron variant of COVID-19).